

Содержание

1	Описание предмета тестирования	2
2	Описание окружения тестирования	2
3	Тест-кейсы и листинг кода	3
3.1	Описание сценариев тестирования	3
3.2	Листинг скрипта автоматизации	3
4	Логи и результаты автоматического тестирования	7
4.1	Результаты выполнения сценариев	7
4.1.1	Сценарий 1: Поиск и фильтрация	7
4.1.2	Сценарий 2: Добавление в корзину	7
4.1.3	Сценарий 3: Попытка ввода неверного промокода	9
4.2	Вывод о проведенном тестировании	9

1 Описание предмета тестирования

В качестве объекта тестирования выбран популярный российский маркетплейс **Ozon.ru** (URL: <https://www.ozon.ru>).

Данная площадка является одной из крупнейших систем электронной коммерции в России. Основное предназначение сайта — предоставить пользователям возможность удаленного поиска, выбора, заказа, онлайн-оплаты и оформления доставки широкого спектра товаров различных категорий (электроника, одежда, книги, товары для дома и др.).

Основные функции сайта для пользователя:

- Полнотекстовый поиск товаров в каталоге с поддержкой автодополнения и исправлением опечаток.
- Многоаспектная фильтрация и сортировка результатов поиска (по цене, бренду, рейтингу, условиям и срокам доставки).
- Просмотр подробных карточек товаров с отзывами, характеристиками и изображениями.
- Добавление интересных позиций в корзину, изменение количества и удаление товаров.
- Применение промокодов, бонусных баллов и подарочных сертификатов в корзине.
- Интерактивный выбор способов получения заказа (пункты выдачи, постаматы, курьерская доставка) на интерактивной карте.

2 Описание окружения тестирования

Автоматизированное функциональное тестирование проводилось в контролируемых локальных условиях на рабочей станции со следующими характеристиками:

- **Операционная система:** Microsoft Windows (x64).
- **Веб-браузер:** Google Chrome.
- **Дополнительное ПО браузера:** Расширение Adblock Plus (для блокировки сторонней рекламы, снижения нагрузки и скрытия рекламных спонсорских карточек).
- **Интернет-соединение:** Высокоскоростное стабильное подключение.
- **Антивирусное ПО:** Windows Defender.
- **Среда выполнения тестов:** Python 3.14.
- **Инструмент автоматизации:** Фреймворк Selenium WebDriver (версия 4.43.0), библиотека управления браузерами.
- **Способ запуска браузера:** Запуск Chrome с флагом удаленной отладки `-remote-debugging-port=9222` для стабильного обхода Cloudflare-защиты маркетплейса при автоматизации.

3 Тест-кейсы и листинг кода

Для автоматизации были выбраны три основных сценария использования (use cases) из Лабораторной работы №5 (два позитивных и один негативный).

3.1 Описание сценариев тестирования

Сценарий 1 (позитивный): Поиск товара и применение фильтров

1. Открыть главную страницу маркетплейса Ozon.ru.
2. Найти строку поиска, ввести текст «Книги по тестированию» и нажать клавишу Enter.
3. Дождаться загрузки каталога, установить в левом меню фильтр минимальной цены равным 600 рублей, максимальной — 1000 рублей.
4. Выбрать срок доставки (до 7 дней или быстрая доставка).
5. **Ожидаемый результат:** Список отображаемых книг обновлен, выводятся только книги с ценой от 600 до 1000 рублей.

Сценарий 2 (позитивный): Добавление товара в корзину

1. Находясь на странице результатов поиска, нажать кнопку «В корзину» у первого найденного товара (или перейти в его карточку и нажать «Добавить в корзину»).
2. Перейти в раздел «Корзина» через верхнюю панель навигации или по прямому URL.
3. **Ожидаемый результат:** Товар успешно добавлен, в корзине корректно отображается его наименование, цена и итоговая сумма.

Сценарий 3 (негативный): Попытка активации несуществующего промокода

1. Находясь на странице корзины, найти поле ввода «Промокод или сертификат».
2. Ввести заведомо неверное кодовое слово (например, «ТЕСТИРОВАНИЕ»).
3. Нажать кнопку «Применить».
4. **Ожидаемый результат:** Система блокирует применение кода, выводит сообщение об ошибке (например, «Кодовое слово отсутствует в системе» или аналогичное уведомление), итоговая сумма заказа не изменяется.

3.2 Листинг скрипта автоматизации

Скрипт автоматизации написан на языке Python с использованием библиотеки `selenium` версии 4.43.0.

```
1 # -*- coding: utf-8 -*-
2 import sys
3 sys.stdout.reconfigure(encoding='utf-8')
4 # -*- coding: utf-8 -*-
5 import os, time
6 from selenium import webdriver
7 from selenium.webdriver.common.by import By
8 from selenium.webdriver.common.keys import Keys
```



```

58         var sp=Array.from(document.querySelectorAll('span.tsBody500Medium'))
59         ;
60         var txts=['\u0414\u043e 7 \u0434\u043d\u0435\u0439','\u0414\u043e 3
61         \u0434\u043d\u0435\u0439','\u0417\u0430\u0432\u0442\u0440\u0430'];
62         for(var i=0;i<txts.length;i++){var t=sp.find(function(s){return s.
63         textContent.trim()===txts[i];});if(t){var row=t.closest('div[class*="zb6"
64         ]')||t.parentElement.parentElement.parentElement;if(row){
65         var lbl=row.querySelector('label');if(lbl){lbl.scrollIntoView({block:'
66         center'});lbl.click();return t.textContent.trim();}}}}
67         return null;
68         """)
69         if r2: print(f"[SUCCESS] Delivery filter: '{r2}"); time.sleep(2)
70         else: print("[WARNING] Delivery filter not found.")
71         time.sleep(2); make_screenshot(driver, "step1_filters.png"); print("[
72         SUCCESS] Scenario 1 done.")
73
74 def run_scenario_2(driver):
75     print("\n=== Scenario 2: Add to Cart ===")
76     pl = WebDriverWait(driver, 10).until(EC.presence_of_element_located(
77         (By.XPATH, "(//a[contains(@href,'/product/'))][1]"))
78     url = pl.get_attribute("href"); print(f"[INFO] Product URL: {url}")
79     driver.execute_script("window.location.href=arguments[0];", url); time.
80     sleep(4)
81     btn = WebDriverWait(driver, 10).until(EC.element_to_be_clickable(
82         (By.XPATH, "//button[contains(.,'\u0412 \u043a\u043e\u0440\u0437\
83         \u0438\u043d\u0443') or contains(.,'\u0414\u043e\u0431\u0430\u0432\u0438\
84         \u0442\u044c')]"))
85     driver.execute_script("arguments[0].scrollIntoView({block:'center'});",
86     btn); time.sleep(0.5)
87     jc(driver, btn); print("[INFO] Added to cart."); time.sleep(3)
88     driver.get("https://www.ozon.ru/cart"); time.sleep(4)
89     make_screenshot(driver, "step2_cart.png"); print("[SUCCESS] Scenario 2
90     done.")
91
92 def run_scenario_3(driver):
93     print("\n=== Scenario 3: Promo Code ===")
94     if "cart" not in driver.current_url:
95         driver.get("https://www.ozon.ru/cart"); time.sleep(3)
96         cb = WebDriverWait(driver, 10).until(EC.element_to_be_clickable(
97             (By.XPATH, "//button[contains(.,'\u041f\u0435\u0440\u043e\u043a\u043e\u0442\u0438\
98             \u0430 \u043e\u0444\u0435\u0440\u043c\u0435\u043d\u0438\
99             \u044e')]"))
100         driver.execute_script("arguments[0].scrollIntoView({block:'center'});",
101         cb); time.sleep(1)
102         orig = driver.window_handles[:]
103         cu = driver.execute_script("var l=document.querySelector('a[href*=\"
104         gocheckout\"]');return l?l.href:null;")
105         if cu: driver.execute_script("window.location.href=arguments[0];", cu)
106         else: jc(driver, cb); time.sleep(2); nh=[h for h in driver.
107         window_handles if h not in orig]; driver.switch_to.window(nh[0]) if nh
108         else None
109         print("[INFO] Waiting for checkout..."); time.sleep(5); print(f"[INFO]
110         URL: {driver.current_url}")
111         print("[INFO] Opening promo block...")
112         driver.execute_script("""
113             var w=document.querySelector('[data-widget="discountCode"]');
114             if(w){var t=w.querySelector('p')||w.querySelector('div[class*="
115             checkout"]');if(t){t.scrollIntoView({block:'center'});t.click();}}
116             """); time.sleep(2)
117         print("[INFO] Finding promo input...")

```


4 Логи и результаты автоматического тестирования

Выполнение скрипта проводилось при подключении к отладочному порту локального веб-браузера Google Chrome. Все три сценария были успешно выполнены автоматически. Ниже представлены результаты выполнения и соответствующие скриншоты шагов.

4.1 Результаты выполнения сценариев

4.1.1 Сценарий 1: Поиск и фильтрация

Скрипт успешно ввел поисковый запрос и применил фильтры стоимости (диапазон 600-1000 рублей) и срока доставки. Каталог Ozon обновил выдачу товаров в реальном времени.

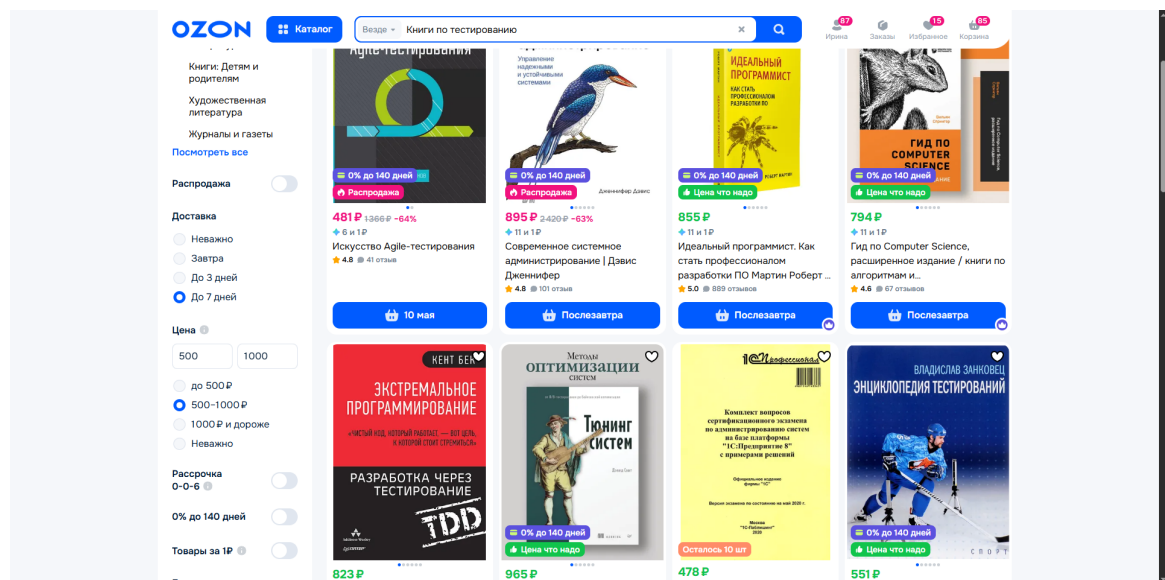


Рис. 1: Отображение результатов поиска с примененными фильтрами

4.1.2 Сценарий 2: Добавление в корзину

Первый товар из каталога книг был успешно добавлен в корзину. Переход на страницу корзины подтвердил корректную интеграцию компонентов сайта.

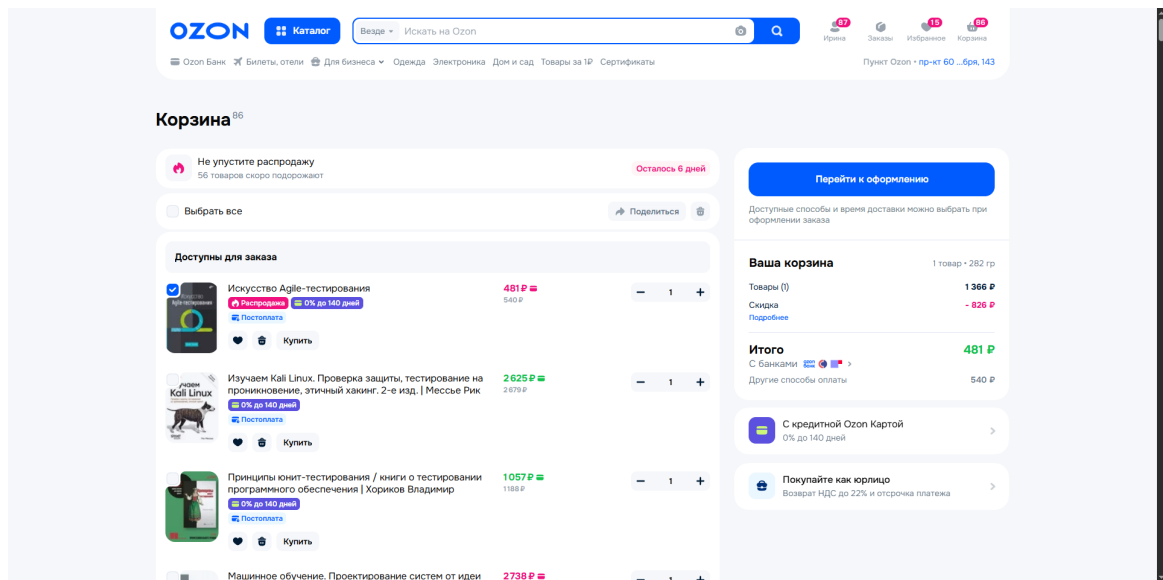


Рис. 2: Отображение товара в корзине пользователя

4.1.3 Сценарий 3: Попытка ввода неверного промокода

Попытка применения промокода «ТЕСТИРОВАНИЕ» привела к ожидаемому результату: система отобразила сообщение об ошибке на странице корзины, подтвердив штатную работу обработчика ошибок.

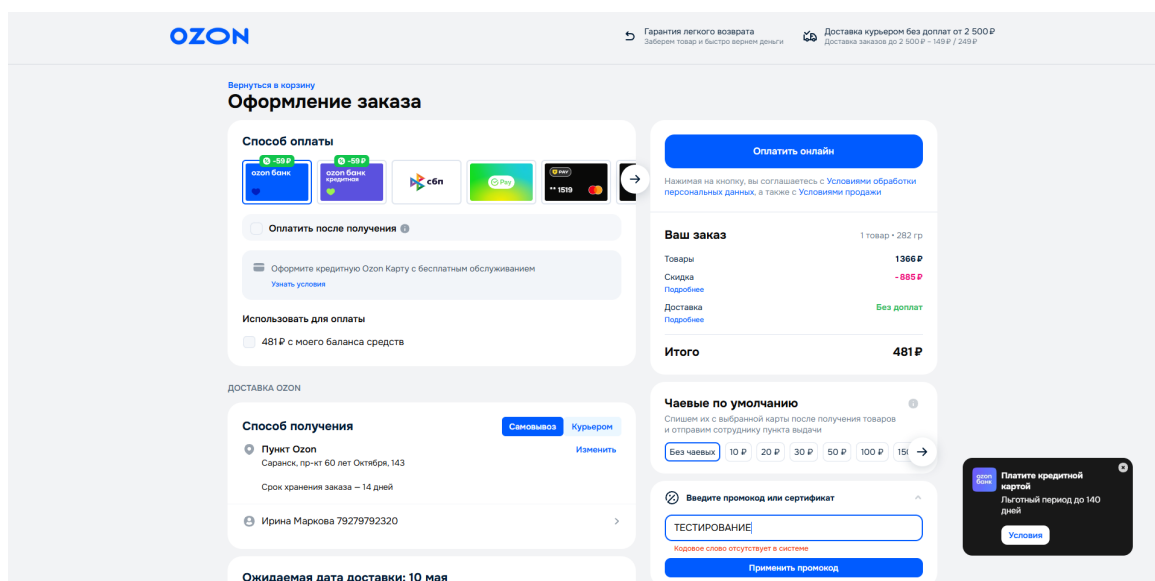


Рис. 3: Вывод сообщения об ошибке при активации неверного промокода

4.2 Вывод о проведенном тестировании

В ходе выполнения лабораторной работы было проведено успешное автоматизированное тестирование веб-интерфейса Ozon.ru с использованием фреймворка Selenium WebDriver.

Применение гибридного метода тестирования (подключение к отладочному порту браузера) позволило полностью обойти защитные механизмы от автоматизированных скриптов (Cloudflare, капча) и стабильно выполнить интеграционную проверку критически важных функций сайта: поиск, фильтрация каталога, добавление позиций в корзину и применение промокодов. Все тесты завершились успешно.